🗺️ Plan → Hacer
el flujo profesional con agentes

Codificar con agentes · Parte 2 · Octubre 2026

👤 Sergi Garcia Barea

CC BY-SA 4.0

⚠️ El límite del "hazlo todo de golpe"

💥 Prompt gigante: "construyeme una app de turnos"

- 8 ficheros absurdos, mitades a medias
- Compila a medias; los tests fallan
- Más caro de corregir que de rehacer
- Nadie entiende qué hizo ni por qué

✅ Prompt por fases: plan + tareas

- Cada paso pequeño y verificable
- Compila y pasa tests en cada iteración
- Tú diriges el rumbo, el agente rema
- El historial de Git explica cada decisión

🎯 La diferencia no es el modelo: es **el flujo de trabajo**. Plan primero, hacer después.

🔊

🗺️ Fase PLAN: preparar el terreno

El agente no ha tocado código todavía. Solo lee, analiza y propone. Es la conversación más barata del proyecto.

🔍 1. Explorar el proyecto

"Lee la estructura y el README. Resúmeme la arquitectura y las convenciones del código."

❓ 2. Pedir opciones y decisiones

"¿Cómo lo estructurarías? ¿Qué alternativas hay? Dime pros y contras de cada una."

📋 3. Fijar requisitos

"Escribe los requisitos como lista con criterios de aceptación. Que sean comprobables."

🧭 4. Aprobar el plan

"Sin código todavía: muéstrame tu plan por pasos. Lo reviso y luego hacemos el paso 1."

💡 En OpenCode, Claude Code o Antigravity basta con decir: **"solo plan, no ejecutes nada"**.

🏗️ Fase HACER: ejecutar paso a paso

🧩 1. Un paso del plan por iteración

"Haz el paso 1 y muéstrame el diff." No el paso 1 y 2 y 3: solo el 1.

🧪 2. Verificar antes de continuar

Compila, corre pruebas, abre la web. El agente puede hacerlo solo: "Comprueba que compila y pasa los tests".

💾 3. Commit con mensaje limpio

"Commit con mensaje explicando el paso. Sigue Conventional Commits." Punto de retorno siempre.

🔁 4. Siguiente paso

Si algo sale mal: "Vuelve atrás y prueba con el enfoque B". El plan es editable.

🎯 Regla del paso pequeño: si no puedes explicar en una frase qué hace la iteración, es demasiado grande.

🗨️ El diálogo completo, en práctica

💬 CONVERSACIÓN CON EL AGENTE

👤 "Añadamos favoritos a la app de recetas.
Solo plan: pasos, ficheros y riesgos."

🤖 "Plan: 1) modelo de datos en localStorage
2) botón corazón en cada receta
3) pantalla 'Mis favoritos'
4) filtros y vacía. Riesgo: duplicados."

👤 "Bien, pero añade en el paso 2 que el botón
también aparezca en la lista. Haz el paso 1."

🤖 (crea store.js + tests) "Compila, 4/4 tests OK."

👤 "Commit. Siguiente paso."

Cuatro mensajes producen una feature completa, verificada y con historial limpio. Sin plan: veinte mensajes de rescate.

🔊

🥅 Criterios de aceptación: saber cuándo está HECHO

📝 PROMPT

```
"Antes de empezar, escribe los criterios
de aceptación de 'favoritos':

- Puedo marcar/desmarcar desde la ficha
  y desde el listado
- Al recargar la página, mis favoritos
  siguen ahí (persistencia)
- La pantalla 'Mis favoritos' muestra
  los marcados, ordenados por fecha
- Compatible con las recetas que ya
  tengo guardadas (sin perder datos)

No programes nada hasta que yo apruebe
la lista."
```

OpenCode · · Criterios antes de código

💡 POR QUÉ FUNCIONA

- ✅ Cada criterio es una prueba que se puede pasar o fallar
- ✅ El agente sabe exactamente cuándo parar
- ✅ Tú verificas contra la lista, contra sensaciones
- ✅ Evita el "te he hecho mucho más" no pedido
- ✅ Es el truco para que no alucine el "done"

🔊

🧱 Dividir en tareas: los prompts que funcionan

✅ Buenas unidades de trabajo

"Añade validación al formulario de registro" · "Extrae la lógica de precio a una función" · "Cubre con tests el módulo de fechas" · "Añade endpoint DELETE para la tarea"

❌ Demasiado grande

"Haz la app completa" · "Refactoriza todo el backend" · "Añade login, perfiles, pagos y notificaciones"

📏 La prueba del tamaño

¿Cambiaría 1–3 ficheros? ¿Se puede verificar compilando y/o con tests? ¿Cabe en un solo mensaje? Entonces tiene el tamaño correcto.

🧭 Orden recomendado

Datos y modelos → lógica + tests → interfaz → detalles visuales → documentación. Cada capa apoya a la siguiente.

🔊

🧪 Verificar: el agente también se autocorrige

📝 PROMPT

```
"Termina el paso 2. Antes de avisarme:

1. Ejecuta npm run test y compila
2. Si algo falla, inténtalo de nuevo
   (máximo 3) leyendo el error real
3. Si sigues atascado, para y explícame
   el error en lugar de dejar
   el código a medias
4. Dime en una línea qué has cambiado"

No toques nada fuera de src/ y tests/."
```

OpenCode · Verificación en bucle

💡 POR QUÉ FUNCIONA

- ✅ El agente lee su propio error y reintenta: de ahí viene buena parte del valor
- ✅ El límite de intentos evita bucles sin fin
- ✅ "Para y explica" es mejor que disimular con código roto
- ✅ El alcance limitado evita ediciones por sorpresa
- ✅ Tú recibes un resumen, no ruido

🔊

🔧 Cuando las cosas van mal

🌀 El agente da vueltas

"Para. Resume lo que has probado y por qué ha fallado. Propón dos enfoques distintos y elige tú el mejor. Solo plan."

🌊 Se pasa de alcance

"Revierte el extra. Haz solo el paso 3 del plan. Lo demás, anótalo como tarea para después."

💥 Rompió algo que funcionaba

"git diff HEAD~1 y dime qué cambió. Restaura lo roto y repite el paso con menos alcance."

🧱 Contexto perdido: sesión nueva

"Lee el README y el git log de los últimos 5 commits. Así es el estado actual." El historial de Git es la memoria del proyecto.

💡 "Para", "revertir" y "solo plan" son tus tres frenos de emergencia. Úsalos sin culpa.

📄 Ejemplo real · ciclo completo

Feature: **importar CSV de alumnos a un sistema de notas**, en 5 pasos verificados.

Paso 1 · Modelo + parser CSV (con errores claros de línea) → tests ✅ → commit

Paso 2 · Validación (DNI, email, nota 0–10) + tests de casos límite → ✅ → commit

Paso 3 · Pantalla de importación con arrastrar-soltar + previsualización → ✅ → commit

Paso 4 · Resultado: importadas / ignoradas con motivos por línea → ✅ → commit

Paso 5 · Botón de plantilla CSV de ejemplo + sección en el README → ✅ → commit

🎯 Cinco commits verificados. Cualquiera puede revertirse sin perder el resto.

🔊

🧘 Modo autónomo con control

Cuando el plan está maduro, puedes dejar que el agente encadene varios pasos. Así se controla.

🤖 Lo que le das

```
"Ejecuta los pasos 2, 3 y 4 del plan
que aprobamos.

Reglas:
- Commit verificado tras cada paso
- Si un paso falla: para y avísame
- No reinventes el plan; si algo
  no encaja, pregunta"
```

🛡️ Lo que te guardas

```
"El plan completo, en un fichero
PLAN.md del repo, actualizado
al terminar cada paso."
```

El plan vive en el repo: cualquier persona (o sesión nueva) lo retoma donde se quedó.

💡 Autonomía sí, a ciegas no: **commits de dos en dos** + plan persistente + un punto de control pactado.

🔊

✨ En esta parte 2

🗺️ PLAN

Explorar · opciones · requisitos con criterios · plan de pasos aprobado — sin tocar código

🏗️ HACER

Un paso por iteración · verificar · commit limpio · iterar — con frenos de emergencia

🥅 **Criterios de aceptación** para saber cuándo está hecho

🧪 **Bucle de verificación**: el agente compila, testea y reintenta antes de avisar

Siguiente: **AGENTS.md y Skills** — cómo enseñarle a trabajar como tú. 📁

CC BY-SA 4.0 · Sergi Garcia Barea · Octubre 2026

🔊